

# Five RSA Attacks: Håstad, Common Modulus, Cycling, Bleichenbacher, and Shared Primes

Every attack below has the same shape: RSA is used with a small mistake in *how it is deployed*, and a tool from Lesson 1 — the Chinese Remainder Theorem, the Extended Euclidean Algorithm, Euler’s theorem, RSA’s multiplicative malleability, or the plain gcd — turns that mistake into a full break. None of them attacks the hardness of factoring itself.

| Attack            | The mistake                               | Tool used            | Fix                             |
|-------------------|-------------------------------------------|----------------------|---------------------------------|
| 1. Håstad         | Same message, small $e$ , many recipients | CRT                  | Randomized padding (OAEP)       |
| 2. Common modulus | One $n$ shared by several users           | Extended Euclid      | Never share $n$                 |
| 3. Cycling        | (none — works in principle on any RSA)    | Euler / group cycles | Large parameters (already true) |
| 4. Bleichenbacher | Server reveals “padding valid?”           | Malleability         | OAEP, uniform errors            |
| 5. Shared prime   | Bad randomness reuses one prime $p$       | gcd (Euclid)         | Good entropy at key generation  |

## 1. Håstad’s Broadcast Attack

### Picture it

Think of  $m^3 \bmod n$  as the reading on a clock with  $n$  hours. If  $m^3$  is bigger than  $n$ , the hand goes around the dial and you only see where it ended up — that wraparound is what hides  $m$ . Now suppose the same  $m^3$  is shown to you on *three different clocks*, of sizes  $n_1, n_2, n_3$ . CRT lets you merge them into one giant clock of size  $n_1 n_2 n_3$ . If  $m^3$  is smaller than that giant clock, the hand never wrapped, so the reading *is*  $m^3$ , and a plain cube root gives  $m$ . Small  $e$  means too little wraparound to hide anything.

**When this arises.** One message must go to several people: a vendor sends the same license key or firmware secret to many customers, or a group announcement is sent encrypted to each member. Every recipient has their own independent key pair, but many use the same tiny exponent (historically  $e = 3$ , chosen because it makes encryption fast on weak hardware). The sender encrypts the identical  $m$  separately under each recipient’s public key.

**Setup.** Exponent  $e = 3$ , the same message  $m$  sent to three recipients with pairwise-coprime moduli

$n_1, n_2, n_3$ . The attacker sees

$$c_i = m^3 \bmod n_i, \quad i = 1, 2, 3.$$

**Theorem 1.** *If  $m < \min(n_1, n_2, n_3)$ , an eavesdropper who sees  $c_1, c_2, c_3$  recovers  $m$  with no private key.*

**Why it works.**

1. CRT combines the three congruences  $x \equiv c_i \pmod{n_i}$  into one unique answer  $C$  modulo  $N = n_1 n_2 n_3$ . This uses only public data.
2. The true value  $m^3$  satisfies all three congruences too, so by uniqueness  $C \equiv m^3 \pmod{N}$ .
3. Because  $m < n_i$  for each  $i$ , we have  $m^3 < n_1 n_2 n_3 = N$ . So  $C$  and  $m^3$  are both in  $[0, N)$  and congruent mod  $N$ , hence *equal as ordinary integers*.
4. Take an ordinary integer cube root of  $C$ . Done.

**Worked example.** Take  $n_1 = 143 = 11 \cdot 13$ ,  $n_2 = 323 = 17 \cdot 19$ ,  $n_3 = 667 = 23 \cdot 29$ , and secret message  $m = 17$ . The three ciphertexts are

$$c_1 = 17^3 \bmod 143 = 51, \quad c_2 = 17^3 \bmod 323 = 68, \quad c_3 = 17^3 \bmod 667 = 244.$$

Each number alone looks unrelated to 17. The attacker now runs CRT with  $N = 143 \cdot 323 \cdot 667 = 30,808,063$ . For each  $i$ , let  $N_i = N/n_i$  and  $y_i = N_i^{-1} \bmod n_i$ :

| $n_i$ | $c_i$ | $N_i$   | $y_i$ |
|-------|-------|---------|-------|
| 143   | 51    | 215,441 | 112   |
| 323   | 68    | 95,381  | 286   |
| 667   | 244   | 46,189  | 221   |

$$C = 51 \cdot 215441 \cdot 112 + 68 \cdot 95381 \cdot 286 + 244 \cdot 46189 \cdot 221 \bmod N = 4913.$$

Since  $17^3 = 4913$  is far less than  $N \approx 3 \times 10^7$ , there was no wraparound at all, and  $\sqrt[3]{4913} = 17 = m$ .

#### Takeaway and fix

Never send the same (or related) message to several recipients under a small  $e$  without randomized padding. With OAEP each ciphertext encrypts a *different* random-looking padded value, so the three equations no longer share one  $m^3$ . (In general, with exponent  $e$  the attacker needs  $e$  recipients.)

## 2. Common Modulus Attack

#### Picture it

This is the classic water-jug puzzle. With a 17-litre jug and a 7-litre jug you can measure exactly 1 litre: fill the 7-litre jug five times and pour out the 17-litre jug twice ( $5 \cdot 7 - 2 \cdot 17 = 1$ ). Here the “jugs” are the exponents. Ciphertext  $c_1$  is  $m$  raised to the power 17, and  $c_2$  is  $m$  raised to the power 7. Multiplying five copies of  $c_2$  and “removing” two copies of  $c_1$  leaves  $m$  raised to the power  $5 \cdot 7 - 2 \cdot 17 = 1$ , that is  $m$  itself.

**When this arises.** Generating a fresh modulus  $n = pq$  for every user is slow, because finding large primes dominates the cost. A careless authority might generate  $n$  once and hand each user a different exponent pair  $(e_i, d_i)$  over that same  $n$ , thinking the different exponents keep users separate. This was a real anti-pattern in some early multi-user and embedded key-management designs.

**Setup.** One shared modulus  $n$ ; two users with  $\gcd(e_1, e_2) = 1$ ; the same message sent to both:

$$c_1 = m^{e_1} \bmod n, \quad c_2 = m^{e_2} \bmod n.$$

**Theorem 2.** If  $\gcd(e_1, e_2) = 1$ , anyone who knows  $(n, e_1, e_2, c_1, c_2)$  recovers  $m$  with no private key.

**Why it works.**

1. Extended Euclid on  $(e_1, e_2)$  gives integers  $a, b$  with  $ae_1 + be_2 = 1$ .
2. Then  $c_1^a c_2^b \equiv m^{ae_1} m^{be_2} = m^{ae_1 + be_2} = m \pmod{n}$ , by exponent laws alone.
3. One of  $a, b$  is negative. For a negative exponent, first compute the modular inverse of the corresponding ciphertext (again by Extended Euclid), then raise to the absolute value.

Notice that the attacker never factors  $n$  and never touches  $d_1$  or  $d_2$ .

**Worked example.** The classic toy key  $n = 3233 = 61 \cdot 53$ , with  $e_1 = 17$ ,  $e_2 = 7$  and secret  $m = 42$ :

$$c_1 = 42^{17} \bmod 3233 = 2557, \quad c_2 = 42^7 \bmod 3233 = 240.$$

*Step 1 — Euclid on  $(17, 7)$ .*  $17 = 2 \cdot 7 + 3$ ,  $7 = 2 \cdot 3 + 1$ . Back-substituting:  $1 = 7 - 2 \cdot 3 = 7 - 2(17 - 2 \cdot 7) = 5 \cdot 7 - 2 \cdot 17$ . So  $a = -2$ ,  $b = 5$ .

*Step 2 — handle the negative exponent.*  $c_1^{-1} \bmod 3233 = 3123$  (check:  $2557 \cdot 3123 \equiv 1$ ).

*Step 3 — combine.*

$$m \equiv (c_1^{-1})^2 \cdot c_2^5 \equiv 2401 \cdot 1904 \equiv 42 \pmod{3233}.$$

**An even worse consequence.** Any *insider* who legitimately owns one pair  $(e_1, d_1)$  on the shared  $n$  can use the fact that  $e_1 d_1 - 1$  is a multiple of  $\varphi(n)$  to factor  $n$ . After that, every other user's private key is compromised too, not just one message.

#### Takeaway and fix

Never share a modulus across users or purposes, even with different exponents. It is broken as soon as two exponents on it are coprime, which is the typical case.

### 3. Cycling Attack (Conceptual)

#### Picture it

Shuffle a deck of cards the same way over and over. Because there are only finitely many orderings, and each shuffle can be undone, you eventually return to the starting order. Whichever ordering you saw *just before* the deck came back was the one you started with. RSA encryption is exactly such a shuffle of the numbers modulo  $n$ : it moves each number  $x$  to  $x^e$ . Keep encrypting a ciphertext and it must eventually return to itself, and the step before that is the plaintext.

**When this arises.** Unlike attacks 1, 2 and 5, this needs *no* mistake in key generation or usage; it applies in principle to any RSA ciphertext. It is not a practical threat, because the cycle is astronomically long for real parameters. It is included because it gives the cleanest picture of what

RSA encryption *is*: iteration of a permutation on a finite set. It also explains why cryptographers historically advised choosing primes so that  $\lambda(n)$  has a large prime factor.

**Setup.** The attacker knows only  $(n, e, c)$  with  $c = m^e \bmod n$ . Re-encrypt repeatedly:

$$c_0 = c, \quad c_1 = c_0^e \bmod n, \quad c_2 = c_1^e \bmod n, \quad \dots$$

until some  $c_k$  equals  $c$  again.

**Theorem 3.** *Let  $k$  be the smallest positive integer with  $c_k = c$ . Then  $m = c_{k-1}$ .*

**Why it works.**

1. The map  $E(x) = x^e \bmod n$  is a bijection on  $\{0, \dots, n-1\}$ , since  $x \mapsto x^d$  undoes it.
2. A bijection on a finite set splits the set into disjoint cycles: starting anywhere and applying  $E$  repeatedly always leads back to the starting point.
3. The plaintext  $m$  lies on one such cycle, and  $c = E(m)$  is the very next element on it. So iterating  $E$  from  $c$  walks around the cycle and returns to  $c$  after some  $k$  steps.
4. The element visited just before returning to  $c$  is the unique element that  $E$  sends to  $c$ , namely  $m$ . So  $c_{k-1} = m$ .

**Worked example.**  $p = 47$ ,  $q = 59$ , so  $n = 2773$ ;  $e = 17$ ; secret  $m = 123$ . The ciphertext is  $c = 123^{17} \bmod 2773 = 1924$ . Now keep raising to the 17th power:

$$1924 \rightarrow 2254 \rightarrow 1280 \rightarrow 1067 \rightarrow 1570 \rightarrow \dots \rightarrow 2224 \rightarrow \boxed{123} \rightarrow 1924.$$

Precisely,  $c_1 = 2254$ ,  $c_2 = 1280$ ,  $\dots$ ,  $c_{42} = 2224$ ,  $c_{43} = 123$ , and  $c_{44} = 1924 = c$ . So  $k = 44$  and  $m = c_{43} = 123$ . It took 44 modular exponentiations against a toy modulus of about 2800.

**Why it stays conceptual.** The cycle length is governed by the multiplicative order of  $e$  modulo  $\text{ord}_n(m)$ , a divisor of  $\lambda(n)$ . For properly generated 2048-bit keys this is far too large to walk, so the attack is infeasible; it would only succeed against deliberately weak parameters.

#### Takeaway and fix

Nothing to patch here: real key sizes already make the cycle unreachable. Use it as a mental model — RSA encryption is stepping around a huge cycle inside a finite group — and as a preview of why the structure of  $\lambda(n)$  (for instance, avoiding smooth  $p-1$ , from Lesson 1) matters.

## 4. Bleichenbacher's Padding Oracle Attack

#### Picture it

Picture the hidden number  $m$  as a runner on a circular track of length  $n$ . A bouncer stands at one narrow gate (the window  $[2B, 3B)$ ) and answers only “yes, someone is in the gate” or “no”. You cannot see  $m$ . But you can make the runner go  $s$  times faster than  $m$  does — that is just multiplying by  $s$  — and ask whether the sped-up runner is now in the gate. A “yes” means the true  $m$  lies in one of a few thin, evenly spaced slices of the track. Choose the next  $s$  so that only one slice remains consistent, repeat, and the slices shrink like a game of Twenty Questions until only  $m$  is left.

**When this arises.** A server decrypts RSA ciphertexts that were padded before encryption, and — by an error message, a dropped connection, or even a timing difference — lets the sender tell

whether the padding was valid. Nobody intends to leak anything else, but it is very easy to do by accident: TLS servers using the PKCS#1 v1.5 padding standard commonly sent a distinguishable “bad padding” alert. This is not history. Refinements (Bardou et al., 2012; the ROBOT attack, 2017) found major production web servers still exploitable nearly twenty years after the original 1998 paper, and it remains one of the most consequential vulnerabilities ever found in a widely deployed protocol.

**What the oracle checks.** PKCS#1 v1.5 formats a  $k$ -byte block as

$$EB = 0x00 \parallel 0x02 \parallel PS \parallel 0x00 \parallel D,$$

where  $D$  is the message and PS is nonzero random padding filling the block to  $k$  bytes. For example, with  $k = 8$ ,  $D = 0x1234$  and  $PS = 0xA1\ 7F\ 3C$ :

$$EB = 00\ 02\ A1\ 7F\ 3C\ 00\ 12\ 34.$$

A **padding oracle** answers one yes/no question about a ciphertext  $c'$  of the attacker’s choosing: *does  $(c')^d \bmod n$  start with 00 02?*

Read as a big integer, “starts with 00 02” simply means the value lies in  $[2B, 3B)$ , where  $B = 256^{k-2}$ . That is a very narrow window: a random ciphertext passes only about 1 time in 65,536. The attack is a way of asking this same narrow question about *cleverly chosen* values instead of random ones.

**The two ingredients.**

- *Malleability.* From  $c = m^e \bmod n$  anyone can compute  $c \cdot s^e \bmod n$  for any chosen  $s$ . This is a valid encryption of  $m \cdot s \bmod n$ , and it needs no secret.
- *A YES is a strong clue.* If the oracle says YES for  $c \cdot s^e$ , then  $ms \bmod n \in [2B, 3B)$  for an  $s$  the attacker knows.

**From one YES to a small set of intervals for  $m$ .** Unwrapping “mod  $n$ ”, a YES for  $s$  means  $ms = rn + t$  with  $t \in [2B, 3B)$  for some unknown whole number  $r$  (how many laps the fast runner made). Dividing by  $s$ :

$$m \in \left[ \frac{2B + rn}{s}, \frac{3B - 1 + rn}{s} \right] \quad \text{for some integer } r.$$

Early on several values of  $r$  are all consistent, so one YES can leave more than one candidate interval (this really happens, as the example shows). Later YES/NO answers eliminate the wrong ones.

**The algorithm in three moves.**

1. **Get started.** If  $c$  is not itself oracle-valid, find a small  $s_0$  with  $c \cdot s_0^e$  oracle-valid and work with  $ms_0$  instead. (If  $c$  is already validly padded,  $s_0 = 1$ .)
2. **Find the next  $s_i$ .** While several intervals survive, try  $s$  upward one at a time until the oracle says YES. Once only one interval  $[a, b]$  survives, jump straight to  $s$  values computed from its endpoints; this roughly doubles  $s$  each round and halves the interval.
3. **Shrink.** For each surviving interval and each possible lap count  $r$ , compute the new sub-interval from the formula above, keep the non-empty ones, and merge overlaps. Stop when one interval containing a single integer remains. That integer is EB (undo  $s_0$  if it was used).

**Worked example (computed and verified).** Toy parameters:  $n = 16,826,323$  ( $p = 4093$ ,  $q = 4111$ ),  $e = 17$ ,  $d = 10,882,313$ , block length  $k = 4$ , so  $B = 256^2 = 65,536$  and the valid window is  $[131,072, 196,608)$ . The secret is  $m = 150,000$  (hex  $0x0249F0$ , which starts  $00\ 02$ , so  $s_0 = 1$ ), with ciphertext  $c = 8,054,451$ .

*The first query in detail.* The attacker tries  $s = 1, 2, 3, \dots$  and the first YES comes at  $s_1 = 450$ . Why? Because

$$450 \cdot 150,000 = 67,500,000 = 4 \cdot n + 194,708,$$

and 194,708 lies in the window. The attacker does not know this, but sees the YES and reasons: “450 $m$  made  $r$  laps and landed in the window.” Two lap counts fit the information so far,  $r = 4$  and  $r = 5$ , giving the two intervals in row 1. The true one is  $r = 4$ .

| $i$ | $s_i$  | surviving interval(s) for $m$             | width                |
|-----|--------|-------------------------------------------|----------------------|
| 0   | —      | [131,072, 196,607]                        | (initial)            |
| 1   | 450    | [149,859, 150,004] and [187,251, 187,396] | two intervals        |
| 2   | 562    | [149,934, 150,004]                        | 70                   |
| 3   | 1,235  | [149,977, 150,004]                        | 27                   |
| 4   | 2,581  | [149,995, 150,004]                        | 9                    |
| 5   | 5,610  | [149,995, 150,002]                        | 7                    |
| 6   | 11,331 | [149,995, 150,000]                        | 5                    |
| 7   | 22,885 | [149,998, 150,000]                        | 2                    |
| 8   | 45,881 | [149,999, 150,000]                        | 1                    |
| 9   | 91,873 | [150,000, 150,000]                        | <b>0 — recovered</b> |

Two things to point out to students. First, the false interval from  $r = 5$  is killed at  $i = 2$ : the next YES ( $s = 562$ ) is consistent only with the true interval. Second, once one interval remains,  $s_i$  roughly doubles each round ( $562 \rightarrow 1235 \rightarrow 2581 \rightarrow 5610 \rightarrow \dots$ ) and the interval width roughly halves: this is binary search. Nine oracle queries against a modulus above sixteen million pinned  $m$  down exactly.

**A caveat about the toy numbers.** Here  $n$  is a 24-bit number, so the block’s leading 00 byte is automatic and the oracle really checks only the second byte (about 1 random value in 256 passes). With a real 2048-bit modulus both bytes must match (about 1 in 65,000), so a real attack needs thousands to about a million queries rather than nine. The mechanics are identical.

#### Takeaway and fix

Never let the reason for a decryption failure be distinguishable — by error content, timing, or connection behavior. The complete fix is to stop using PKCS#1 v1.5 encryption in new designs and use OAEP (Lesson 3), whose security proof rules out this whole class of attack.

## 5. Shared-Prime (Batch GCD) Attack

#### Picture it

Factoring a single 2048-bit number is like trying to un-bake a cake: hopeless. But if two cakes were baked with a common ingredient, you can find it just by comparing them — and Euclid’s algorithm does that comparison in a fraction of a millisecond. If two RSA moduli happen to share a prime, computing  $\gcd(n_1, n_2)$  hands you that prime, and dividing gives the other prime of each. Both keys are broken, and neither owner did anything obviously wrong.

**When this arises.** Devices such as routers, firewalls and other embedded systems often generate their RSA key the first time they boot. At that moment the operating system’s random number generator has had almost no entropy (no disk, no keyboard, no network jitter yet), so many devices

produce *the same* first prime  $p$ . A little entropy trickles in (a clock tick, a packet) before the second prime is chosen, so their  $q$ 's differ. The result is many public keys of the form  $n_i = p q_i$  with a common  $p$ .

This is not hypothetical. In 2012 two independent teams (Lenstra et al., “*Ron was wrong, Whit is right*”; Heninger et al., “*Mining your Ps and Qs*”) collected millions of public RSA keys from the internet and factored thousands of them this way, with no ciphertext and no private key needed — just public moduli. To avoid comparing every pair, they used “batch GCD”, a product-tree algorithm that checks millions of moduli against each other in hours.

**Setup.** The attacker has two public moduli

$$n_1 = p q_1, \quad n_2 = p q_2, \quad q_1 \neq q_2,$$

and knows nothing else about the factorizations.

**Theorem 4.** *Then  $\gcd(n_1, n_2) = p$ . Consequently  $q_1 = n_1/p$  and  $q_2 = n_2/p$ , and the private keys of both moduli can be computed.*

**Why it works.**

1.  $p$  divides both  $n_1$  and  $n_2$ , so  $p \mid \gcd(n_1, n_2)$ .
2. Since  $q_1$  and  $q_2$  are distinct primes (and differ from  $p$ ),  $\gcd(n_1, n_2) = p \cdot \gcd(q_1, q_2) = p$ .
3. With all primes known, compute  $\varphi(n_1) = (p-1)(q_1-1)$  and  $d_1 = e_1^{-1} \bmod \varphi(n_1)$  — exactly the recipe the legitimate key owner used. The same works for  $n_2$ .

The gcd takes  $O(\log^2 n)$  bit operations, whereas factoring a 2048-bit modulus directly is infeasible. That gap is the entire attack.

**Worked example.** Alice’s key is  $n_1 = 3233 = 61 \cdot 53$  and Bob’s key is  $n_2 = 3599 = 61 \cdot 59$ ; both use  $e = 17$ . Eve sees only the two public moduli, so all she starts with is 3233 and 3599.

*Step 1 — Euclid.*

$$\begin{aligned} 3599 &= 1 \cdot 3233 + 366, \\ 3233 &= 8 \cdot 366 + 305, \\ 366 &= 1 \cdot 305 + 61, \\ 305 &= 5 \cdot 61 + 0, \end{aligned}$$

so  $\gcd(3233, 3599) = 61$ . Eve has found the shared prime  $p = 61$  in four division steps.

*Step 2 — factor both keys.*  $q_1 = 3233/61 = 53$  and  $q_2 = 3599/61 = 59$ .

*Step 3 — rebuild Alice’s private key.*  $\varphi(n_1) = 60 \cdot 52 = 3120$  and  $d_1 = 17^{-1} \bmod 3120 = 2753$  (check:  $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$ ). Bob’s key falls the same way:  $\varphi(n_2) = 60 \cdot 58 = 3480$ ,  $d_2 = 1433$ .

*Step 4 — read a message.* Suppose someone sent Alice the ciphertext  $c = 2183$ . Eve computes  $2183^{2753} \bmod 3233 = 1234$ , the plaintext, without ever being given a key.

#### Takeaway and fix

Generate keys only after the system has gathered enough entropy, use the operating system’s cryptographic random number generator, and draw  $p$  and  $q$  from fresh randomness. Avoid generating keys on the first boot of a device with no entropy source. Operators can also check their own newly generated keys against large public key collections for shared factors. Notice the lesson: RSA’s security depends not only on the math but on the quality of the random numbers used to pick the primes.

---

**Where each tool came from.** All five attacks reuse Lesson 1 material as *attack* tools rather than construction tools: CRT (attack 1), the Extended Euclidean Algorithm (attack 2), Euler's theorem (attack 3), multiplicative malleability (attack 4), and the plain Euclidean gcd (attack 5). Attacks 1, 2 and 5 exploit a *misuse*; attack 4 exploits an *information leak*; attack 3 is a purely structural observation about RSA.